Placement-Based Double Deep Q-Network for Sample-Efficient Tetris Policy Learning

Khang Le Duy Dang[1], Cao-Phuc Ha[1], and Phu-Nguyen Le[2(✉)]

[1] Department of Artificial Intelligence, FPT University, Ho Chi Minh City, Vietnam
{khang17122005,caophucai}@gmail.com
[2] Faculty of Electrical and Electronics Engineering, Vietnam Aerospace University, 104 Nguyen Van Troi Street, Phu Nhuan Ward, Ho Chi Minh City 700000, Vietnam
nguyenlp@vaa.edu.vn

Abstract. Learning Tetris policies from interaction remains difficult. Pixel-based deep reinforcement learning suffers from sparse and delayed rewards, and the strongest controllers still rely on handcrafted evaluation functions. This paper proposes P-DDQN, a placement-based Double Deep Q-Network. The agent scores every legal final placement of the current tetromino with a four-dimensional afterstate descriptor: aggregate height, holes, bumpiness, and cleared lines. A multilayer perceptron with 4,545 parameters estimates afterstate values, and a periodically synchronized target network separates placement selection from value estimation to reduce overestimation bias. Within 1,050 training episodes, completed in 16.8 min on a single central processing unit (CPU), the agent reaches a single-game record of 1,127 cleared lines. In a 100-game benchmark capped at 1,000 placements, P-DDQN clears 287.5 ± 131.8 lines per game, 15.3 times more than a naive greedy controller. The learned policy favours multi-line clears and earns 81.3 points per cleared line. These results exceed those of all published learning-based agents in the comparison, while training takes only minutes on commodity hardware.

Keywords: Deep reinforcement learning · Double DQN · Tetris · Afterstate · Sample efficiency.

1 Introduction

Tetris is a stochastic sequential decision problem. Each placement permanently changes the board and constrains all later moves. Even the offline version, in which the whole piece sequence is known, is NP-complete [1]. The 20×10 grid also admits up to 2^{200} occupancy configurations, so exact dynamic programming is infeasible. For these reasons, Tetris has long served as a benchmark for approximate dynamic programming and reinforcement learning (RL) [2, 3].

Existing learning approaches face three limitations. (1) Vision-based deep RL, such as the Deep Q-Network (DQN) [4], operates on raw frames with step-wise micro-actions; its rewards are sparse and delayed, and millions of interactions are needed. (2) Many learned controllers depend on large simulation budgets

or on training with graphics processing units (GPUs), which makes them hard to reproduce on commodity hardware. (3) Handcrafted evaluation functions use fixed expert weights that are not learned from interaction [5, 6].

To address these limitations, this paper proposes a placement-based Double Deep Q-Network (P-DDQN). The agent enumerates all legal final placements (x, r) of the current tetromino. For each resulting board, it extracts four geometric features: aggregate height, hole count, bumpiness, and cleared lines. A multilayer perceptron (MLP) with 4,545 parameters predicts the value of each board, and the agent selects the placement with the highest value. Learning targets come from a periodically synchronized target network, following the Double DQN principle [7]. Experiments use a 20×10 board and a 7-bag piece generator. Within 1,050 training episodes, completed in 16.8 min on a single central processing unit (CPU), the agent reaches a record of 1,127 cleared lines. In a 100-game benchmark, it clears 287.5 lines per game on average, 15.3 times more than a naive greedy controller. The learned policy also favours multi-line clears and earns 81.3 points per cleared line.

The remainder of this paper is organized as follows. Section 2 reviews learning-based and handcrafted approaches to Tetris. Section 3 describes the placement action space, the afterstate features, the reward design, and the Double DQN learning procedure. Section 4 presents the simulator, the baselines, and the implementation details. Section 5 reports the learning dynamics, the benchmark results, the line-clearing behaviour, and the comparison with published agents. Section 6 concludes the paper and outlines future work.

2 Related Work

Tetris is a well-studied benchmark for value-function approximation and policy search [3]. Early feature-based value iteration used only two features, the wall height and the number of holes, and cleared 32 rows per game [8]. Temporal-difference (TD) policy iteration over column heights, height differences, maximum height, and holes reached 3,183 lines [9]. Least-squares policy iteration reported 1,000–3,000 points per game [10], and approximate linear programming with randomized constraint sampling reported 4,274 lines [11]. Direct policy search achieved much higher counts. The noisy cross-entropy method [12], the covariance matrix adaptation evolution strategy (CMA-ES) [13], cross-entropy over engineered features [14], and classification-based policy iteration [15] learn linear evaluation functions that clear from hundreds of thousands to tens of millions of lines. Handcrafted functions, such as the controller of Pierre Dellacherie documented by Fahey [5] and the four-weight function of Lee [6], also sustain very long games. However, these methods depend on engineered feature sets and fixed linear models, or on very large evaluation budgets. Evolutionary training of Tetris players, for example, can take weeks on a desktop computer [13].

Several learning-based agents with relational, case-based, or imitation-based representations obtained much lower line counts. Relational reinforcement learning (RRL) with first-order regression trees and guidance traces cleared about

16 lines per game after 10,000 episodes [16]. Its Gaussian-process variant with graph kernels reached at most 43 lines per game within 2,000 s of training [17]. Case-based reasoning (CBR) with local board patterns, valued by TD(0) updates, cleared 44.1 lines per game over 100 test games [18]. The Training an Agent Manually via Evaluative Reinforcement (TAMER) framework learns a linear model from human feedback instead of the game reward and reached 65.89 lines per game [19]. Supervised backpropagation networks that imitate Dellacherie's controller completed only 3 rows per game on the full seven-piece game [20]. These results show that learning a competent evaluation function from interaction remains difficult, even with compact representations.

DQN reached human-level control on many Atari games from raw pixels [4], but this approach transfers poorly to Tetris. Stevens and Pradhan trained a convolutional neural network (CNN) on downsampled game screens [21]. Learning did not converge with single-move actions. Grouping moves into final placements and shaping the reward with a heuristic raised the average game score, computed with squared-line scoring, to 18.0. Hendriks trained a TD-learning MLP over column-height features and reached a running average of about 23 lines per game after about 40 h of training on a laptop [22]. Liu and Liu evaluated DQN, Proximal Policy Optimization (PPO), data-regularized Q-learning (DrQ), and Dreamer agents on Nintendo Entertainment System (NES) Tetris [23]. None of these pixel-based agents cleared a line in testing after several days of training, whereas a DQN over hand-crafted features performed well in a simplified placement environment. Chen combined feature-based deep Q-learning with reward shaping, a hold action, and expected-value updates; the best configuration required 12 h of training [24]. Yan et al. applied PPO with dynamic timesteps and transferred the policy to a robotic arm [25]. Bairaktaris and Johannssen showed that a heuristic agent outperformed DQN, categorical DQN (C51), and PPO agents on Atari Tetris in both score and computational efficiency [26]. Livada et al. found that a standard DQN failed to clear lines in Tetris even after about 17 h of CPU training [27]. Most recently, Antonietto and Silva combined Double DQN with afterstates and a variable reward function and obtained long survival times [28].

In summary, pixel-based formulations suffer from long credit-assignment delays. The successful feature-based deep agents rely on richer feature sets, extra game mechanics, or hours of training, and few studies report their computational budget. The proposed framework addresses these gaps. It combines macro placement actions, a minimal four-dimensional afterstate descriptor, and a 4,545-parameter Double DQN that trains in 16.8 min on a single CPU. It also analyses the multi-line clearing behaviour that this formulation induces.

3 Proposed Methodology

The proposed framework consists of four stages (Fig. 1). First, all legal placements of the current tetromino are enumerated. Second, afterstate features are extracted for each placement. Third, an online network selects the placement

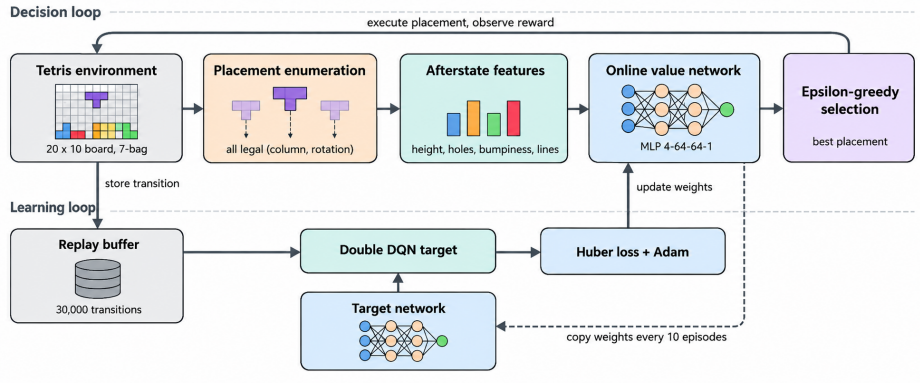


Fig. 1. Overview of the proposed P-DDQN framework: placement enumeration, afterstate feature extraction, value-based ϵ-greedy selection, and Double DQN learning with experience replay and a periodically synchronized target network.

with the highest value. Fourth, Double DQN learns from an experience replay buffer with a periodically synchronized target network.

3.1 Problem Formulation and Placement Action Space

Tetris is modelled as a Markov decision process [29]. The state consists of the binary 20×10 board and the current tetromino, and the random arrival of the next piece drives the transitions. Instead of step-wise shifts and rotations, each action $a = (x, r)$ selects an orientation r of the current tetromino and the column x where it is hard-dropped. The seven tetrominoes have 19 distinct orientations, so at most 34 placements are legal in any state. Each decision therefore spans a single time step, which shortens the credit-assignment horizon from dozens of micro-actions to one placement [21].

3.2 Afterstate Feature Representation

A placement is deterministic once the current piece is known. Each action a therefore yields a known afterstate $s'_t = T(s_t, a)$, namely the board after the piece lands and full rows are removed. Learning afterstate values avoids modelling the random next piece [29]. Each afterstate is described by the four geometric features popularized by Lee [6]. The aggregate height f_1 is the sum of column heights. The hole count f_2 is the number of empty cells covered by at least one occupied cell in the same column. The bumpiness f_3 is the sum of absolute height differences between adjacent columns. The last feature $f_4 \in \{0, \ldots, 4\}$ is the number of lines completed by the placement. The descriptor $\phi(s'_t) = [f_1, f_2, f_3, f_4]^\top$ thus compresses up to 2^{200} board configurations into four numbers that govern survival.

3.3 Reward Design

The scalar reward balances line clearing against board stability:

$$r_t = 1 + 10 f_4^2 - 0.5\,\Delta f_1 - 1.5\,\Delta f_2 - 0.5\,\Delta f_3, \tag{1}$$

where Δf_i is the change of feature i between consecutive afterstates. The constant term rewards survival. The quadratic term gives a multi-line clear a larger reward than the same number of single clears. The difference terms penalize increases in height, holes, and roughness. A terminal penalty of -10 is applied when the next tetromino cannot be placed.

3.4 Value Network and Double DQN Learning

The value network is an MLP with two hidden layers of 64 rectified linear units (ReLUs). It maps $\phi \in \mathbb{R}^4$ to a scalar afterstate value $V(\phi;\theta)$ and has 4,545 trainable parameters with Kaiming-uniform initialization. Because an action is fully characterized by its afterstate, the agent selects $a_t = \arg\max_a V(\phi(T(s_t,a));\theta)$ with ϵ-greedy exploration. Standard Q-learning uses the same parameters to select and to evaluate the maximizing action, which overestimates action values [7]. Following Double DQN, the successor afterstate ϕ_{t+1} is selected by the online parameters θ and evaluated by periodically synchronized target parameters θ^-:

$$y_t = r_t + \gamma\,(1-d_t)\,V\left(\phi_{t+1};\theta^-\right), \tag{2}$$

where d_t marks a terminal transition. Transitions $(\phi_t, r_t, \phi_{t+1}, d_t)$ are stored in a cyclic replay buffer [30]. The online network minimizes the Huber loss [31] between y_t and $V(\phi_t;\theta)$ over uniformly sampled mini-batches. The target parameters θ^- are copied from θ every $C = 10$ episodes. Because ϕ_{t+1} is chosen with the same ϵ-greedy rule used for acting, the target equals the greedy Double DQN target once ϵ has decayed.

4 Experimental Setup

4.1 Environment

Experiments use a purpose-built Tetris simulator with a 20×10 board. The seven standard tetrominoes are drawn from a 7-bag randomizer, which shuffles all seven pieces before each refill. Each action hard-drops the current piece in the chosen orientation and column. Hold, piece preview, and soft-drop moves are not used. Completed rows are removed immediately, and a game ends when the next tetromino has no legal placement. The score follows the classic single-level rule of 40, 100, 300, and 1,200 points for clearing one, two, three, and four lines.

4.2 Baselines

Two non-learning baselines use the same simulator and placement actions. The *random* policy selects a legal placement uniformly at random. The *naive greedy* controller scores each afterstate by $-f_1 + f_4$. It therefore minimizes aggregate height and maximizes immediate line clears, but ignores holes and bumpiness.

4.3 Implementation Details

P-DDQN is implemented in PyTorch and trained on a single consumer CPU without a GPU. Optimization uses Adam [32] with a learning rate of 10^{-3}, a mini-batch size of 256, and a discount factor $\gamma = 0.99$. One gradient step is taken per placement, and the gradient norm is clipped at 1.0. The replay buffer holds 30,000 transitions. The exploration rate ϵ decays linearly from 1.0 to 0.001 over the first 1,000 episodes. The checkpoint with the highest training line count is kept for evaluation. Each agent is evaluated greedily ($\epsilon = 0$) over 100 games, with a ceiling of 1,000 placements per game. All agents play the same 100 piece sequences, which are generated from fixed random seeds. The reported metrics are the mean and standard deviation of cleared lines, the median and maximum lines, the mean score, and the mean number of placements (survival).

5 Results

5.1 Learning Dynamics

Fig. 2 summarizes the training run. During the exploration phase ($\epsilon > 0.4$, episodes 1–600), games last about 30 placements and rarely clear a line. As ϵ decreases, the value network starts to exploit the learned board geometry. The mean rises from 4.3 lines in episodes 601–800 to 15.0 lines in episodes 801–900 and to 51.5 lines in episodes 901–1000. The first game with at least 100 cleared lines occurs at episode 975. After exploration has decayed, the agent clears 288.2 lines per episode on average in episodes 1001–1050. The single-game record of 1,127 lines is reached at episode 1,010, after only 60,440 cumulative placements. The full run of 1,050 episodes uses 88,352 placements and 16.8 min of wall-clock time.

5.2 Benchmark Performance

Table 1 and Fig. 3(a) report the 100-game benchmark. The random policy survives 25.5 placements and clears 0.1 lines per game, so the placement action space alone does not produce competent play. The naive greedy controller clears 18.8 ± 13.6 lines. Because it ignores holes, it tends to bury empty cells under low surfaces and ends after 85.2 placements on average. P-DDQN clears 287.5 ± 131.8 lines per game (median 388.0, minimum 9, maximum 398) and survives 749.1 placements on average. This is 15.3 times more cleared lines and 8.8 times longer survival than the naive greedy controller, and 29.4 times longer survival than random placement. P-DDQN reaches the 1,000-placement ceiling in 54 of the 100 games, so its mean line count is limited by the evaluation budget.

5.3 Line-Clearing Behaviour

The points-per-line ratio in Table 1 and Fig. 3(b) reveals the learned playing style. Under the scoring rule, a single-line clear yields 40 points per line, whereas a

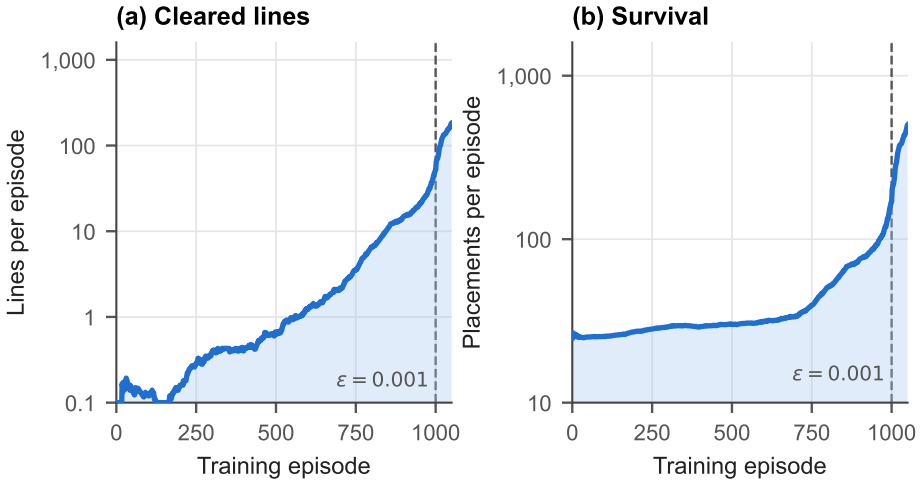


Fig. 2. Training dynamics of P-DDQN over 1,050 episodes, shown as 100-episode moving averages on a logarithmic scale: (a) cleared lines per episode and (b) placements per episode. The dashed line marks episode 1,000, where ϵ reaches 0.001.

Table 1. Benchmark results over 100 games per agent with a ceiling of 1,000 placements. Bold indicates the proposed method.

	Cleared lines			Mean	Mean	Points
Agent	Mean ± std	Median	Max	score	placements	per line
Random	0.1 ± 0.3	0.0	1	4.8	25.5	40.0
Naive greedy	18.8 ± 13.6	16.5	66	965.4	85.2	51.5
P-DDQN	$\mathbf{287.5 \pm 131.8}$	**388.0**	**398**	**23,360.8**	**749.1**	**81.3**

four-line clear yields 300 points per line. The naive greedy controller obtains 51.5 points per line, so it clears lines mostly one at a time. P-DDQN obtains 81.3 points per line. The quadratic line term of the reward drives the network to keep the surface level and to delay clears until several rows can be removed at once. This behaviour also explains the higher variance of P-DDQN. Delaying clears raises the stack for a while, which increases both the attainable score and the risk of early termination.

5.4 Computational Profile

The value network has 4,545 parameters. The full checkpoint, including the online network, target network, and optimizer state, occupies 82 KB on disk. A placement decision enumerates and evaluates at most 34 candidate afterstates and takes about 5.4 ms on a standard CPU, which allows more than 180 decisions per second. The 1,050-episode training run of Sect. 5.1 takes 16.8 min on

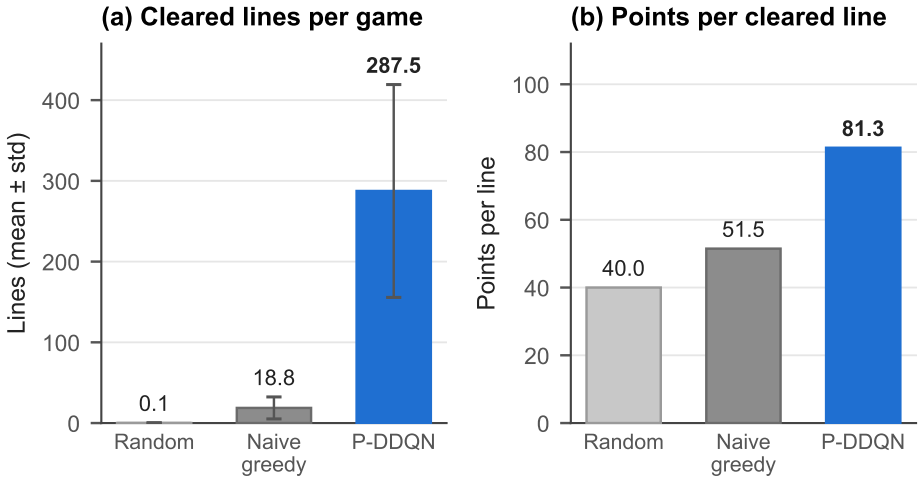


Fig. 3. Evaluation over 100 games per agent: (a) cleared lines per game (mean ± standard deviation) and (b) points earned per cleared line. A value of 40 points per line corresponds to single-line clears only.

the same CPU without a GPU. The framework is therefore easy to reproduce and suitable for embedded, real-time use.

5.5 Comparison with Related Methods

Table 2. Comparison with published learning-based Tetris agents, ordered by mean cleared lines per game. Values of prior work are taken from the original publications; values marked with ≈ or ≤ are read from published learning curves. Evaluation games are the games averaged in the reported mean, and the gain is the ratio between the mean lines of P-DDQN and those of each study.

Study	Lines per game	Evaluation games	Training episodes	Gain of P-DDQN (×)
Knox & Stone [19]	65.89	9	3	4.4
Romdhane & Lamontagne [18]	44.1	100	>500	6.5
Driessens et al. [17]	≤43	—	≈5,000	≥6.7
Tsitsiklis & Van Roy [8]	32	100	—	9.0
Hendriks [22]	≈23	100	3,000	12.5
Driessens & Džeroski [16]	≈16	500	10,000	18.0
P-DDQN (proposed)	**287.5**	**100**	**1,050**	**1.0**

Table 2 compares P-DDQN with learning-based Tetris agents from the literature. Direct comparison is limited by differences in protocol, such as board

height, termination rule, piece generator, and number of evaluation games. All values of prior work are taken from the original publications. The strongest prior agent in the table, TAMER, learns from human feedback and reaches 65.89 lines per game [19]. P-DDQN exceeds this result by a factor of 4.4 without any human feedback. It also clears 6.5 times more lines than CBR with TD(0)-valued patterns [18], at least 6.7 times more than RRL with Gaussian processes [17], and 9.0 times more than feature-based value iteration [8]. Compared with the TD-learning MLP of Hendriks, P-DDQN clears 12.5 times more lines and trains in 16.8 min instead of about 40 h [22]. Supervised imitation of an expert controller completes only 3 rows per game [20], which suggests that learning from value targets is preferable to imitating expert moves. Deep agents that operate on pixels or raw grids did not clear lines, even after several days [23] or about 17 h of training [27]. The best pixel-based configuration of Stevens and Pradhan reaches an average game score of 18.0 under squared-line scoring [21]. These results indicate that placement actions and a compact geometric state are the main source of sample efficiency. The Double DQN target complements this design by limiting overestimation in the value estimates [7].

6 Conclusion

This paper presented P-DDQN, a placement-based Double Deep Q-Network for sample-efficient Tetris policy learning. The method combines macro placement actions, a four-dimensional afterstate representation, and decoupled target evaluation. A network with 4,545 parameters reached a training record of 1,127 lines within 1,050 episodes, which took 88,352 placements and 16.8 min on a single CPU. In a 100-game evaluation, it cleared 287.5 lines per game on average, 15.3 times more than a naive greedy controller. The learned policy favours multi-line clears and earns 81.3 points per cleared line, with a mean score of 23,361 under a fixed 1,000-placement budget. Future work will evaluate longer, uncapped games over several training seeds. It will also enrich the afterstate descriptor and apply the placement formulation to related packing and scheduling problems, such as two-dimensional bin packing.

Acknowledgments. The authors would like to express their sincere appreciation to the Vietnam Aerospace University for supporting this research during the 2026–2027 academic year.

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Demaine, E.D., Hohenberger, S., Liben-Nowell, D.: Tetris is hard, even to approximate. In: Warnow, T., Zhu, B. (eds.) COCOON 2003. LNCS, vol. 2697, pp. 351–363. Springer, Heidelberg (2003). https://doi.org/10.1007/3-540-45071-8_36

2. Bertsekas, D.P., Tsitsiklis, J.N.: Neuro-Dynamic Programming. Athena Scientific, Belmont, MA (1996)
3. Algorta, S., Şimşek, Ö.: The game of Tetris in machine learning. arXiv preprint arXiv:1905.01652 (2019). https://doi.org/10.48550/arXiv.1905.01652
4. Mnih, V., Kavukcuoglu, K., Silver, D., et al.: Human-level control through deep reinforcement learning. Nature **518**(7540), 529–533 (2015). https://doi.org/10.1038/nature14236
5. Fahey, C.P.: Tetris AI, computer plays Tetris (2003). http://colinfahey.com/tetris/tetris_en.html, last accessed 2026/09/30
6. Lee, Y.: Tetris AI – the (near) perfect bot. Code My Road (2013). https://codemyroad.wordpress.com/2013/04/14/tetris-ai-the-near-perfect-player/, last accessed 2026/09/30
7. van Hasselt, H., Guez, A., Silver, D.: Deep reinforcement learning with double Q-learning. In: Proceedings of the AAAI Conference on Artificial Intelligence, vol. 30, pp. 2094–2100. AAAI Press (2016). https://doi.org/10.1609/aaai.v30i1.10295
8. Tsitsiklis, J.N., Van Roy, B.: Feature-based methods for large scale dynamic programming. Machine Learning **22**, 59–94 (1996). https://doi.org/10.1007/BF00114724
9. Bertsekas, D.P., Ioffe, S.: Temporal differences-based policy iteration and applications in neuro-dynamic programming. Technical Report LIDS-P-2349, Massachusetts Institute of Technology, Cambridge, MA (1996)
10. Lagoudakis, M.G., Parr, R., Littman, M.L.: Least-squares methods in reinforcement learning for control. In: Vlahavas, I.P., Spyropoulos, C.D. (eds.) SETN 2002. LNCS (LNAI), vol. 2308, pp. 249–260. Springer, Heidelberg (2002). https://doi.org/10.1007/3-540-46014-4_23
11. Farias, V.F., Van Roy, B.: Tetris: A study of randomized constraint sampling. In: Calafiore, G., Dabbene, F. (eds.) Probabilistic and Randomized Methods for Design Under Uncertainty, pp. 189–201. Springer, London (2006). https://doi.org/10.1007/1-84628-095-8_6
12. Szita, I., Lőrincz, A.: Learning Tetris using the noisy cross-entropy method. Neural Computation **18**(12), 2936–2941 (2006). https://doi.org/10.1162/neco.2006.18.12.2936
13. Boumaza, A.: On the evolution of artificial Tetris players. In: 2009 IEEE Symposium on Computational Intelligence and Games (CIG), pp. 387–393. IEEE (2009). https://doi.org/10.1109/CIG.2009.5286451
14. Thiery, C., Scherrer, B.: Building controllers for Tetris. ICGA Journal **32**(1), 3–11 (2009). https://doi.org/10.3233/ICG-2009-32102
15. Gabillon, V., Ghavamzadeh, M., Scherrer, B.: Approximate dynamic programming finally performs well in the game of Tetris. In: Advances in Neural Information Processing Systems, vol. 26, pp. 1754–1762 (2013)
16. Driessens, K., Džeroski, S.: Integrating guidance into relational reinforcement learning. Machine Learning **57**(3), 271–304 (2004). https://doi.org/10.1023/B:MACH.0000039779.47329.3a
17. Driessens, K., Ramon, J., Gärtner, T.: Graph kernels and Gaussian processes for relational reinforcement learning. Machine Learning **64**(1–3), 91–119 (2006). https://doi.org/10.1007/s10994-006-8258-y
18. Romdhane, H., Lamontagne, L.: Reinforcement of local pattern cases for playing Tetris. In: Wilson, D., Lane, H.C. (eds.) Proceedings of the Twenty-First International Florida Artificial Intelligence Research Society Conference (FLAIRS 2008), pp. 263–268. AAAI Press, Menlo Park, CA (2008)

19. Knox, W.B., Stone, P.: Interactively shaping agents via human reinforcement: the TAMER framework. In: Proceedings of the Fifth International Conference on Knowledge Capture (K-CAP 2009), pp. 9–16. ACM, New York (2009). https://doi.org/10.1145/1597735.1597738
20. Lewis, I.J., Beswick, S.L.: Generalisation over details: the unsuitability of supervised backpropagation networks for Tetris. Advances in Artificial Neural Systems **2015**, 157983 (2015). https://doi.org/10.1155/2015/157983
21. Stevens, M., Pradhan, S.: Playing Tetris with deep reinforcement learning. CS231n course project report, Stanford University (2016). https://cs231n.stanford.edu/reports/2016/pdfs/121_Report.pdf
22. Hendriks, G.: The effect of state representation in reinforcement learning applied to Tetris. Bachelor's thesis, University of Groningen (2020). https://fse.studenttheses.ub.rug.nl/22952/
23. Liu, H., Liu, L.: Learn to play Tetris with deep reinforcement learning. IERG5350 course project report, The Chinese University of Hong Kong (2020). https://openreview.net/forum?id=8TLyqLGQ7Tg
24. Chen, Z.: Playing Tetris with deep reinforcement learning. Master's thesis, University of Illinois at Urbana-Champaign (2021). http://hdl.handle.net/2142/110682
25. Yan, Y., Liu, P., Zhao, J., Zhang, C., Wang, G.: Deep reinforcement learning in playing Tetris with robotic arm experiment. Transactions of the Institute of Measurement and Control **47**(12), 2333–2342 (2025). https://doi.org/10.1177/01423312221114694
26. Bairaktaris, J.A., Johannssen, A.: Outsmarting algorithms: A comparative battle between reinforcement learning and heuristics in Atari Tetris. Expert Systems with Applications **277**, 127251 (2025). https://doi.org/10.1016/j.eswa.2025.127251
27. Livada, Č., Duka, M., Keser, T., Nenadić, K.: Deep Q-network agents for game playing: systematic evaluation across eight benchmark and custom environments. Electronics **15**(5), 958 (2026). https://doi.org/10.3390/electronics15050958
28. Antonietto, A.M., Silva, F.: Solving Tetris through double deep Q-learning using afterstates and a variable reward function. In: Intelligent Systems (BRACIS). LNCS, pp. 446–460. Springer, Cham (2026). https://doi.org/10.1007/978-3-032-39882-6_30
29. Sutton, R.S., Barto, A.G.: Reinforcement Learning: An Introduction, 2nd edn. MIT Press, Cambridge, MA (2018)
30. Lin, L.J.: Self-improving reactive agents based on reinforcement learning, planning and teaching. Machine Learning **8**(3–4), 293–321 (1992). https://doi.org/10.1007/BF00992699
31. Huber, P.J.: Robust estimation of a location parameter. The Annals of Mathematical Statistics **35**(1), 73–101 (1964). https://doi.org/10.1214/aoms/1177703732
32. Kingma, D.P., Ba, J.: Adam: A method for stochastic optimization. In: 3rd International Conference on Learning Representations (ICLR) (2015). https://doi.org/10.48550/arXiv.1412.6980